ECOLE MAROCAINE DES SCIENCES DE L'INGENIEUR
Membre de HONORIS UNITED UNIVERSITIES

Rapport de Projet - Blockchain & Cryptographie

MediChain+

Plateforme blockchain de santé numérique

Ilyas AJELYAN
Anas EL KARKOURI
Omar BABBA
Omar GAGA
Amin ER-REBBAT

4ÈME ANNÉE INGÉNIERIE INFORMATIQUE ET RÉSEAUX (4IIR)

Année universitaire : 2025 - 2026

Encadrant à l'EMSI :
Pr. Hassan BADIR

ECOLE MAROCAINE DES SCIENCES DE L'INGENIEUR
Membre de HONORIS UNITED UNIVERSITIES

Remerciements

Nous tenons à exprimer notre reconnaissance à **Pr. Hassan BADIR**, encadrant du module Blockchain & Cryptographie, pour son suivi, ses remarques et son exigence méthodologique. Ses orientations ont aidé l'équipe à dépasser une simple démonstration technique pour construire une solution structurée, argumentée et vérifiable.

Nous remercions l'École Marocaine des Sciences de l'Ingénieur pour le cadre académique offert aux projets appliqués. Le module a permis d'aborder la blockchain sous un angle concret : comprendre la cryptographie, les registres distribués, les smart contracts, l'identité numérique et leurs limites dans un domaine sensible comme la santé.

Nous remercions également les membres de l'équipe **MediChain+** pour leur implication. La réalisation du projet a demandé une coordination entre plusieurs domaines : **Hyperledger Fabric**, **Solidity**, **IPFS**, **API Gateway**, frontend **React**, sécurité, tests et documentation. Cette complémentarité a permis de produire une plateforme démontrable et cohérente.

Enfin, nous remercions toutes les personnes qui ont donné des retours pendant la préparation de la démonstration. Ces remarques ont contribué à améliorer la lisibilité du rapport, la clarté des écrans, la justification des choix techniques et la qualité globale du livrable final.

Résumé

MediChain+ est une plateforme de santé numérique conçue dans le cadre du module Blockchain & Cryptographie. Le projet répond à une problématique réelle : les données médicales sont souvent dispersées entre plusieurs acteurs, les remboursements d'assurance restent lents et peu transparents, et la traçabilité pharmaceutique manque parfois de preuves vérifiables.

La solution proposée repose sur une architecture hybride. **Hyperledger Fabric** est utilisé pour les flux privés : patients, consentements, dossiers médicaux et prescriptions. La couche **EVM**, exécutée localement avec **Hardhat** et pensée pour **Polygon**, porte les smart contracts de remboursement et de traçabilité. **IPFS Kubo** sert au stockage distribué des fichiers chiffrés, tandis qu'une **API Gateway Express** coordonne l'ensemble et applique **JWT**/**RBAC**.

L'application web **React** fournit quatre portails : Patient, Médecin, Pharmacie et Assureur. Chaque portail correspond à un rôle métier et à des permissions spécifiques. Le patient gère son dossier et ses remboursements, le médecin crée les dossiers et prescriptions, la pharmacie vérifie et dispense, et l'assureur valide puis exécute les remboursements via smart contract.

Le projet a été exécuté localement pour produire les captures intégrées au rapport. Les validations réalisées confirment la compilation des contrats, 47 tests automatisés réussis, un build frontend fonctionnel et une **API Gateway** opérationnelle. **IPFS** était configuré mais le daemon **Kubo** n'était pas lancé pendant la capture ; cette limite est donc signalée explicitement.

Mots clés : blockchain hybride, santé numérique, **Hyperledger Fabric**, **Polygon**, **Solidity**, **Hardhat**, **IPFS**, chiffrement **AES-256-GCM**, **SHA-256**, **DID/SSI**, **JWT**, **RBAC**, remboursement, traçabilité pharmaceutique.

Abstract

MediChain+ is a digital healthcare platform developed for the Blockchain & Cryptography module. The project addresses a realistic problem: medical data is fragmented across multiple stakeholders, insurance reimbursement is slow and difficult to audit, and pharmaceutical traceability often lacks verifiable evidence.

The proposed solution uses a hybrid architecture. **Hyperledger Fabric** handles private workflows such as patients, consents, medical records and prescriptions. The **EVM** layer, executed locally with **Hardhat** and designed for **Polygon** compatibility, hosts reimbursement and drug traceability smart contracts. **IPFS Kubo** stores encrypted off-chain files, while an **Express API Gateway** coordinates the components and enforces **JWT**/**RBAC** controls.

The **React** web application provides four portals: Patient, Doctor, Pharmacy and Insurer. Each portal maps to a business role and a set of permissions. The implementation was launched locally to capture the screenshots included in this report. The validation phase confirmed contract compilation, 47 passing automated tests, a successful frontend build and an operational **API Gateway**. **IPFS** was configured but not running during the capture, which is documented as a limitation.

Keywords: hybrid blockchain, digital healthcare, **Hyperledger Fabric**, **Polygon**, **Solidity**, **Hardhat**, **IPFS**, **AES-256-GCM**, **SHA-256**, **DID/SSI**, **JWT**, **RBAC**, reimbursement, pharmaceutical traceability.

Table des matières

Figures 7
Tableaux 7
Chapitre 1 : Problématique, objectifs et solution proposée 10
1.1 Contexte métier 10
1.2 Problématique 10
1.3 Objectifs du projet 11
1.4 Solution proposée 11
1.5 Valeur ajoutée de MediChain+ 12
1.6 Périmètre et limites assumées 13
1.7 Synthèse du chapitre 13
Chapitre 2 : Analyse fonctionnelle et spécification 14
2.1 Acteurs 14
2.2 Besoins fonctionnels 14
2.3 Exigences non fonctionnelles 15
2.4 Scénarios principaux 16
2.5 Critères de réussite 16
2.6 Synthèse du chapitre 17
Chapitre 3 : Conception technique et architecture 18
3.1 Vue d'ensemble 18
3.2 Séparation des données 19
3.3 API Gateway 19
3.4 Smart contracts 20
3.5 Séquence de remboursement 20
3.6 Conception de la sécurité 21
3.7 Analyse des Risques et Sécurité 21
Chapitre 4 : Technologies utilisées et réalisation 23
4.1 Frontend React et Vite 23
4.2 API Gateway Express 23
4.3 Hyperledger Fabric 23

4.4 Solidity, Hardhat et OpenZeppelin 24
4.5 IPFS, chiffrement et hachage 24
4.6 Correspondance technologies / responsabilités 24
4.7 Organisation de l'équipe 25
Chapitre 5 : Tests, résultats et validation 26
5.1 Environnement exécuté 26
5.2 Captures de l'application lancée 26
5.3 Portail Patient 27
5.4 Portail Médecin 28
5.5 Portail Pharmacie 30
5.6 Portail Assureur 30
5.7 Résultats de validation 31
5.8 Indicateurs quantitatifs 32
5.9 Analyse des résultats obtenus 33
5.10 Perspectives 33

Liste des figures et des tableaux

Figures

Figure 1. Workflow Fonctionnel Global de **MediChain+**

Figure 2. Carte problème-solution de **MediChain+**

Figure 3. Diagramme de cas d'utilisation de la plateforme

Figure 4. Architecture hybride de **MediChain+**

Figure 5. Séparation des données et flux de sécurité

Figure 6. Diagramme de séquence du remboursement

Figure 7. Interface principale après activation du mode Démo

Figure 8. Portail Patient exécuté localement

Figure 9. Portail Médecin exécuté localement

Figure 10. Portail Pharmacie exécuté localement

Figure 11. Portail Assureur exécuté localement

Tableaux

Tableau 1. Liste des abréviations

Tableau 2. Besoins fonctionnels par acteur

Tableau 3. Exigences non fonctionnelles

Tableau 4. Analyse des risques et mesures de mitigation

Tableau 5. Correspondance technologies et responsabilités

Tableau 6. Résultats de validation technique

Tableau 7. Indicateurs quantitatifs de validation

Tableau 8. Comparaison du stockage on-chain et hors chaîne

Liste des abréviations

Les abréviations suivantes sont utilisées dans les chapitres techniques du rapport.

Tableau 1. Liste des abréviations

Abréviation	Signification
API	Application Programming Interface
CID	Content Identifier utilisé par IPFS pour identifier un contenu
DID/SSI	Decentralized Identifier / Self-Sovereign Identity
EVM	Ethereum Virtual Machine
IPFS	InterPlanetary File System
JWT	JSON Web Token
RBAC	Role-Based Access Control
SHA-256	Algorithme de hachage utilisé pour vérifier l'intégrité
AES-256-GCM	Chiffrement symétrique authentifié pour protéger les fichiers

Introduction Générale

La transformation numérique du secteur médical repose de plus en plus sur des systèmes interconnectés. Un patient peut consulter un médecin, recevoir une prescription, acheter un médicament, transmettre une preuve à son assureur et attendre un remboursement. Pourtant, ces opérations sont souvent réparties entre plusieurs applications qui ne partagent pas toujours le même niveau de confiance, de sécurité et de traçabilité.

Cette fragmentation crée plusieurs difficultés. Les dossiers médicaux peuvent être incomplets ou difficiles à vérifier. Les consentements du patient ne sont pas toujours visibles pour tous les acteurs autorisés. Les remboursements peuvent nécessiter des contrôles manuels lents. Enfin, la traçabilité des médicaments peut manquer d'un historique fiable, surtout lorsque plusieurs organisations interviennent dans la chaîne.

La blockchain apporte des mécanismes utiles à cette problématique : immutabilité, horodatage, hachage, signature numérique, smart contracts et auditabilité. Toutefois, le domaine médical impose une contrainte essentielle : les données personnelles de santé ne doivent pas être exposées publiquement. Une solution crédible doit donc distinguer les données sensibles, les preuves techniques et les règles automatisées.

MediChain+ répond à cet enjeu par une architecture hybride. Le projet combine **Hyperledger Fabric** pour les échanges privés, une couche **EVM** pour les smart contracts, **IPFS** pour le stockage chiffré hors chaîne, une **API Gateway** pour l'orchestration et une interface **React** pour la démonstration des usages. Cette combinaison permet de montrer les concepts du cours tout en respectant les contraintes métier.

La problématique générale du projet peut être formulée ainsi : comment concevoir une plateforme de santé numérique qui protège les données médicales, conserve des preuves vérifiables, automatise certains traitements et reste démontrable dans un contexte académique ?

Pour y répondre, ce rapport présente d'abord la problématique et les objectifs, puis l'analyse fonctionnelle, la conception technique, les technologies utilisées, la réalisation, les tests et les résultats observés. Les captures d'écran intégrées proviennent de l'application exécutée localement, afin de documenter l'état réel du projet.

Chapitre 1 : Problématique, objectifs et solution proposée

Objectif du chapitre

Ce chapitre clarifie le besoin métier, la problématique centrale du projet, les objectifs fonctionnels et la réponse globale apportée par **MediChain+**.

1.1 Contexte métier

Dans un parcours médical classique, plusieurs acteurs manipulent des informations liées au même patient. Le médecin produit un diagnostic et une prescription, la pharmacie vérifie puis dispense un traitement, et l'assureur contrôle les éléments nécessaires au remboursement. Lorsque ces opérations ne sont pas reliées par une couche de confiance commune, chaque acteur conserve sa propre version des faits.

La conséquence directe est une perte de fluidité. Le patient peut devoir répéter les mêmes informations, le médecin peut manquer d'historique, la pharmacie peut avoir du mal à vérifier l'authenticité d'une prescription, et l'assureur peut demander plusieurs justificatifs avant de valider une claim. Le problème n'est donc pas seulement technique ; il concerne aussi la coordination et la confiance entre organisations.

1.2 Problématique

Problématique retenue

Comment garantir la confidentialité des données médicales tout en rendant les opérations critiques vérifiables, traçables et partiellement automatisables entre patient, médecin, pharmacie et assureur ?

Cette problématique impose un compromis. Une blockchain publique est utile pour publier des preuves ou exécuter des règles transparentes, mais elle n'est pas adaptée au stockage direct d'informations médicales. À l'inverse, une base privée protège mieux les données, mais elle ne fournit pas naturellement les garanties d'audit et d'immutabilité attendues d'un registre distribué.

MediChain+ évite cette opposition en séparant les responsabilités. Les données sensibles restent dans des couches contrôlées : **Fabric** pour les métadonnées privées, **IPFS** chiffré pour les fichiers et l'API pour les accès. Les smart contracts **EVM** sont réservés aux traitements qui nécessitent une preuve d'état ou une automatisation, notamment le cycle de remboursement et la traçabilité des lots.

1.3 Objectifs du projet

- Sécuriser la gestion des patients, des consentements, des dossiers médicaux et des prescriptions.
- Automatiser le remboursement d'assurance selon un cycle vérifiable : soumission, validation et paiement.
- Conserver une trace lisible des lots de médicaments et de leurs changements de statut.
- Démontrer l'application concrète des notions de hachage, chiffrement, signatures, identité numérique, smart contracts et immutabilité.
- Fournir une application présentable avec quatre portails métiers et un mode Démo exploitable pendant une soutenance.

1.4 Solution proposée

La solution proposée est une plateforme web multi-portails. Le patient gère son profil, ses consentements et ses remboursements. Le médecin crée les dossiers médicaux et les prescriptions. La pharmacie vérifie les prescriptions, dispense les médicaments et consulte la traçabilité. L'assureur valide les claims, alimente le contrat et exécute les paiements.

Le projet ne cherche pas à remplacer tout le système hospitalier. Il présente plutôt un prototype académique complet qui montre comment des briques blockchain peuvent être combinées avec une application web classique. L'intérêt pédagogique est important : chaque composant correspond à une notion étudiée dans le module.

Le workflow fonctionnel global ci-dessous donne une lecture métier de bout en bout. Il montre comment une demande part du patient, passe par le médecin, sépare les métadonnées privées et les fichiers chiffrés, puis aboutit à la validation assurantielle et au paiement automatisé.

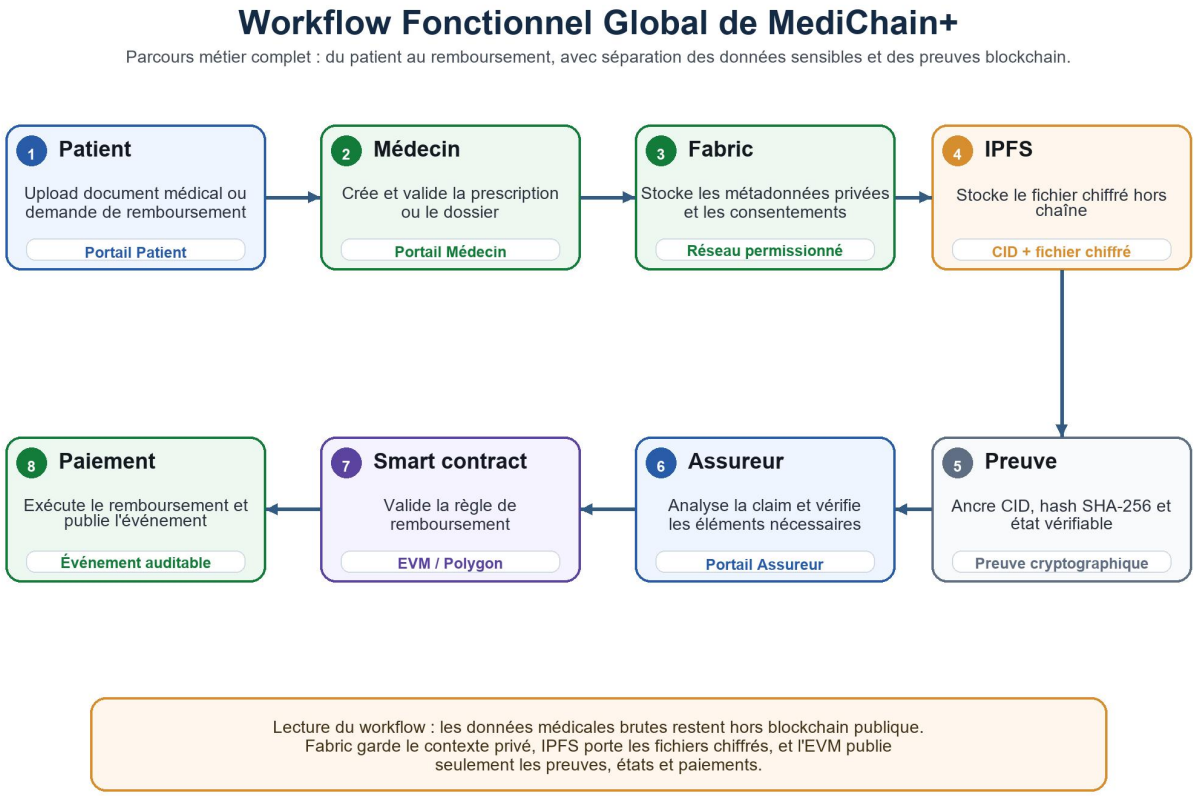


Figure 1. Workflow Fonctionnel Global de MediChain+.

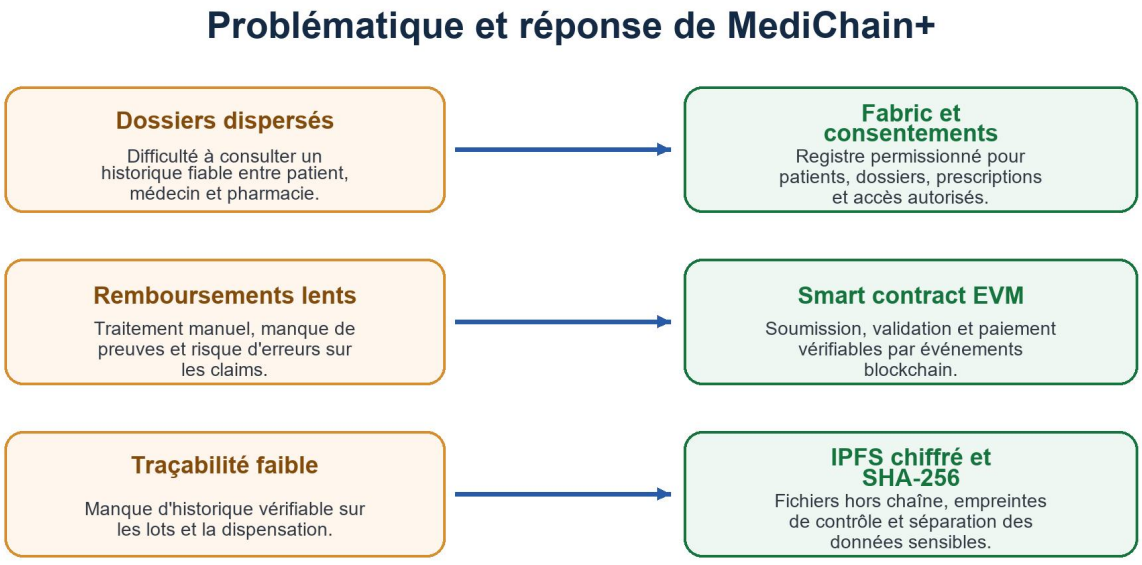


Figure 2. Carte problème-solution de MediChain+.

1.5 Valeur ajoutée de MediChain+

La valeur ajoutée de **MediChain+** se situe dans l'intégration cohérente de plusieurs technologies. Beaucoup de prototypes blockchain se limitent à un smart contract

isolé ou à une interface sans logique métier réelle. Ici, le projet relie l'identité, les rôles, les dossiers, les prescriptions, le stockage chiffré, les claims de remboursement et la traçabilité pharmaceutique.

Le deuxième apport est la séparation claire entre confidentialité et auditabilité. Les données médicales n'ont pas vocation à être exposées publiquement, mais les événements critiques doivent rester vérifiables. **MediChain+** répond à ce compromis avec **Hyperledger Fabric**, **IPFS**, **SHA-256** et les smart contracts **EVM**.

Le troisième apport est la démonstrabilité. Le mode Démo, la barre d'état, les quatre portails et les tests automatisés permettent de présenter le projet sans dépendre d'une configuration fragile. Cette dimension est essentielle dans un contexte académique, car elle transforme l'architecture en expérience observable.

1.6 Périmètre et limites assumées

Le périmètre couvre la démonstration locale : frontend **React**, **API Gateway**, **Fabric** en mode mock ou réseau **Fabric** selon l'environnement, nœud **Hardhat** local pour l'**EVM**, contrats **Solidity**, et intégration **IPFS** préparée. Les captures du rapport proviennent du frontend lancé en local.

La limite principale observée pendant la capture concerne **IPFS** : le client est configuré, mais le daemon **Kubo** n'était pas actif. Le rapport signale donc cette limite au lieu de présenter une validation non réalisée. Pour une version industrielle, il faudrait aussi renforcer la gouvernance des clés, les règles réglementaires, la supervision et le déploiement multi-organisations.

1.7 Synthèse du chapitre

Ce premier chapitre établit la logique générale de **MediChain+** : résoudre un problème de confiance entre plusieurs acteurs de santé sans exposer les données médicales brutes. Le workflow global montre que la valeur du projet vient moins d'une blockchain isolée que d'une orchestration complète entre patient, médecin, pharmacie, assureur et services techniques.

Cette base fonctionnelle prépare les chapitres suivants. L'analyse fonctionnelle précise les rôles et scénarios, la conception technique explique la séparation **Fabric**/**IPFS**/**EVM**, et la validation démontre que ces choix sont implémentés dans une application exécutable.

Chapitre 2 : Analyse fonctionnelle et spécification

Objectif du chapitre

Ce chapitre décrit les acteurs, les besoins fonctionnels, les exigences non fonctionnelles et les scénarios principaux que l'application doit couvrir.

2.1 Acteurs

Le patient est l'acteur central. Il consulte ses informations, gère les consentements et soumet une demande de remboursement. Son rôle impose une interface claire et un contrôle strict de la confidentialité.

Le médecin produit des données médicales : dossiers, diagnostics et prescriptions. Ses actions doivent être tracées car elles déclenchent ensuite la dispensation en pharmacie et peuvent justifier un remboursement.

La pharmacie intervient au moment de la vérification et de la dispensation. Elle consulte les prescriptions, vérifie leur statut et participe à la traçabilité pharmaceutique.

L'assureur contrôle les claims et exécute le remboursement. Il doit pouvoir consulter les événements, valider une demande et payer uniquement lorsqu'une claim respecte les règles prévues.

L'administrateur ou l'équipe technique configure les contrats, les rôles, certains droits de mise à jour et les services nécessaires à la démonstration.

2.2 Besoins fonctionnels

Tableau 2. Besoins fonctionnels par acteur

Acteur	Besoins principaux
Patient	Créer ou consulter un profil, gérer les consentements, consulter prescriptions et dossiers, soumettre une claim de remboursement.
Médecin	Créer un dossier médical, créer une prescription, rechercher les dossiers autorisés, vérifier l'intégrité d'un record.
Pharmacie	Consulter une prescription, vérifier une signature, dispenser le médicament, suivre l'historique d'un lot.

Acteur	Besoins principaux
Assureur	Consulter les claims, valider une demande, provisionner le contrat, exécuter le paiement, lire les événements.
Système	Vérifier les rôles, journaliser les opérations, séparer les données privées, calculer des empreintes et exposer l'état des services.

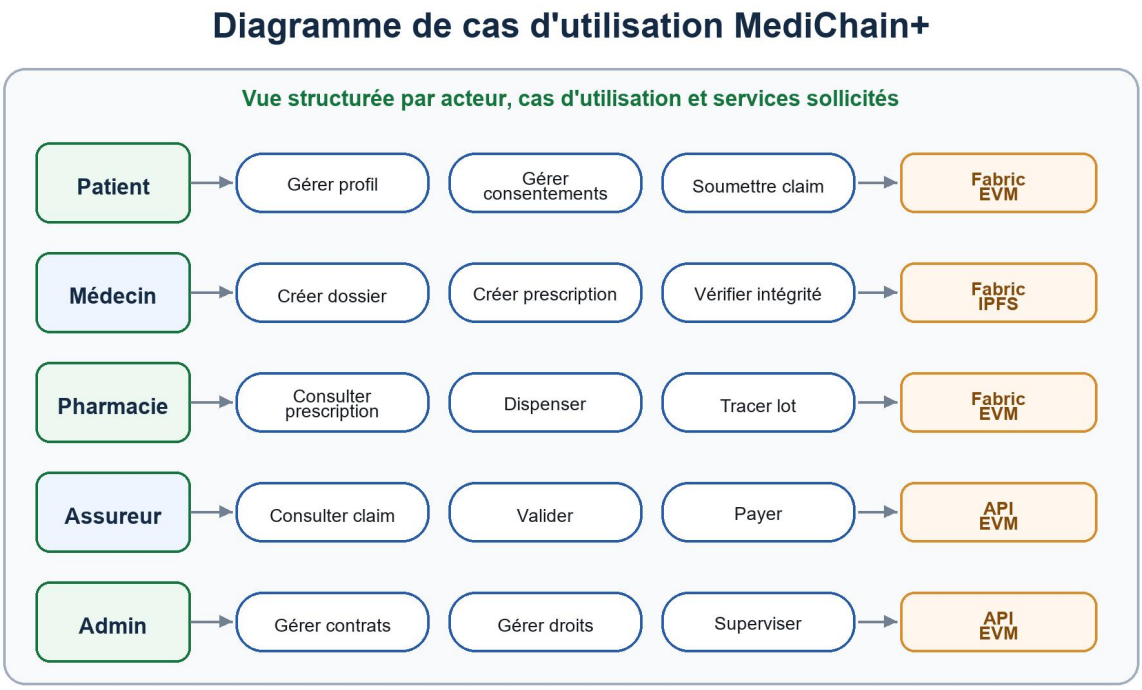


Figure 3. Diagramme de cas d'utilisation de la plateforme.

2.3 Exigences non fonctionnelles

Tableau 3. Exigences non fonctionnelles

Exigence	Interprétation dans MediChain+
Confidentialité	Ne pas exposer les données médicales brutes sur une blockchain publique ; utiliser Fabric, IPFS chiffré et RBAC.
Intégrité	Vérifier les contenus par hachage SHA-256 et conserver des événements qui rendent les opérations auditables.
Traçabilité	Conserver l'historique des prescriptions, dossiers, claims et lots de médicaments.

Exigence	Interprétation dans MediChain+
Disponibilité de démonstration	Permettre une présentation avec mode Démo même sans MetaMask ; afficher l'état API, Fabric, EVM, IPFS.
Maintenabilité	Séparer frontend, API, contrats, Fabric, IPFS et tests afin de faciliter la compréhension et l'évolution.
Sécurité applicative	Contrôler les rôles, valider les paramètres, limiter les abus, protéger les routes et tester les accès interdits.

2.4 Scénarios principaux

Scénario patient : l'utilisateur ouvre l'application, active le mode Démo ou connecte son wallet, consulte l'état des services, accède au portail Patient, puis soumet une demande de remboursement associée à un montant et à un contexte médical.

Scénario médecin : le médecin consulte un patient autorisé, crée un dossier médical, génère une prescription et vérifie l'intégrité d'un record. Les métadonnées sont prises en charge par **Fabric** et les preuves peuvent être reliées aux mécanismes cryptographiques du projet.

Scénario pharmacie : la pharmacie consulte une prescription, vérifie sa signature ou son statut, exécute la dispensation et interroge la traçabilité d'un lot. Ce flux montre l'intérêt de conserver un historique append-only.

Scénario assureur : l'assureur consulte le solde du contrat, lit une claim, la valide, exécute le paiement et vérifie les événements blockchain. Ce scénario démontre l'usage concret du smart contract **ReimbursementContract**.

2.5 Critères de réussite

- L'application doit être lançable localement avec un frontend accessible et des portails fonctionnels.
- Les contrats doivent compiler et les tests automatisés critiques doivent réussir.
- Les captures du rapport doivent provenir du projet lancé, pas d'images génériques ou d'anciens fichiers.
- Les schémas doivent expliquer la vraie architecture : **Fabric**, **API Gateway**, **EVM**, **IPFS** et frontend.
- Le rapport doit rester lisible après impression : police claire, paragraphes justifiés, figures larges et absence de pages vides inutiles.

2.6 Synthèse du chapitre

L'analyse fonctionnelle montre que **MediChain+** ne se limite pas à une expérimentation blockchain abstraite. Les besoins sont liés à des rôles métiers identifiables et à des opérations concrètes : créer un dossier, vérifier une prescription, dispenser un médicament, soumettre une claim et payer un remboursement.

Cette analyse prépare la conception technique. Chaque besoin impose une responsabilité architecturale : **Fabric** pour les données privées, **EVM** pour l'automatisation vérifiable, **IPFS** pour les fichiers chiffrés, **API Gateway** pour l'orchestration et **React** pour la démonstration des usages.

Chapitre 3 : Conception technique et architecture

Objectif du chapitre

Ce chapitre présente les choix de conception, la séparation des responsabilités, les flux de sécurité et la séquence de remboursement.

3.1 Vue d'ensemble

L'architecture de **MediChain+** repose sur cinq couches. La couche présentation est développée en **React** avec **Vite** et **TypeScript**. La couche orchestration est une **API Gateway Express**. La couche privée est représentée par **Hyperledger Fabric**. La couche smart contracts repose sur **Solidity**, **Hardhat** et une compatibilité **Polygon**/**EVM**. La couche stockage distribué utilise **IPFS Kubo** avec chiffrement des fichiers.

Cette séparation rend la solution plus compréhensible et plus réaliste. **Fabric** est utilisé là où les participants doivent être connus et les données protégées. **EVM** est utilisé là où une règle doit être vérifiable et automatisable. **IPFS** évite de placer des fichiers volumineux dans les registres. L'**API Gateway** applique l'authentification, les rôles et les validations métier.

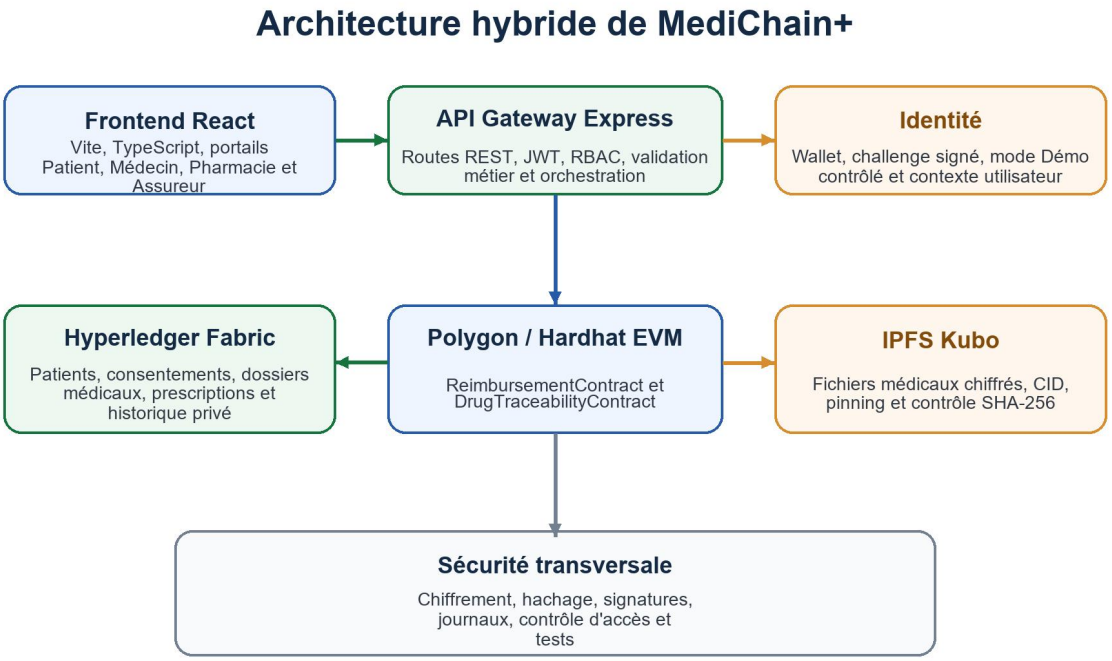


Figure 4. Architecture hybride de MediChain+.

3.2 Séparation des données

Les dossiers médicaux ne sont pas stockés tels quels dans une blockchain publique. La conception sépare les contenus médicaux, les métadonnées, les preuves et les événements. Les métadonnées privées sont gérées par **Fabric** ; les fichiers peuvent être chiffrés et placés sur **IPFS** ; les smart contracts stockent des états minimaux utiles au remboursement et à la traçabilité.

Cette séparation limite l'exposition des données sensibles. Même si un événement **EVM** est visible, il ne doit pas contenir le contenu médical. Le lien avec les dossiers passe par des identifiants, des empreintes ou des références contrôlées. Cette approche est indispensable dans un contexte de santé.

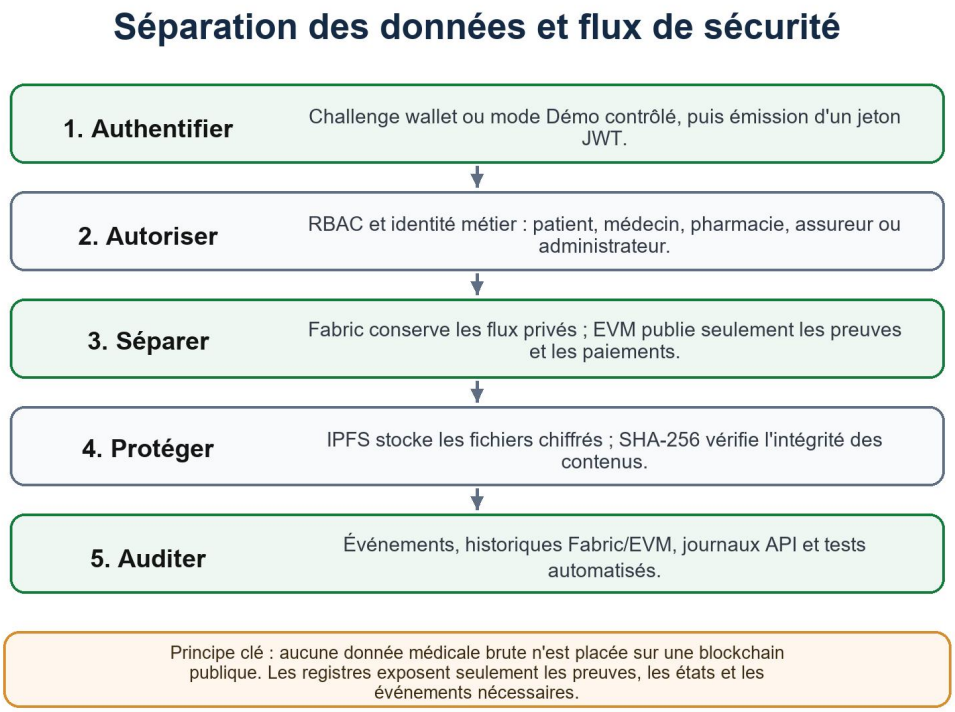


Figure 5. Séparation des données et flux de sécurité.

3.3 API Gateway

L'**API Gateway** expose les routes utilisées par le frontend : authentification, patients, consentements, dossiers, prescriptions, **IPFS**, remboursements et médicaments. Elle vérifie les jetons **JWT**, applique les rôles et transmet les opérations aux composants concernés.

Les middlewares observés ajoutent des en-têtes de sécurité, limitent les requêtes abusives, contrôlent le bearer token et vérifient les rôles. Les fonctions d'autorisation distinguent admin, patient, doctor, pharmacy et insurer. Cette couche est critique car elle évite que le frontend manipule directement des ressources sensibles.

3.4 Smart contracts

ReimbursementContract modélise une demande de remboursement. Une claim contient un identifiant, l'adresse du patient, un montant, un état d'approbation et un état de paiement. Le contrat refuse les montants nuls, les claims inexistantes, les doubles validations, les paiements sans approbation et les paiements répétés.

Le paiement est protégé par ReentrancyGuard. Le contrat utilise Ownable afin que les opérations sensibles soient réservées au propriétaire, représentant l'autorité d'assurance ou le gateway autorisé. La fonction submitClaimFor permet à l'API de soumettre une claim pour le wallet patient authentifié.

DrugTraceabilityContract modélise un lot pharmaceutique par batchId. Le contrat conserve le nom du médicament, le fabricant, l'existence du lot et un historique de statuts. Les mises à jour sont append-only : l'historique s'enrichit mais ne réécrit pas les anciennes entrées.

3.5 Séquence de remboursement

Le remboursement illustre l'articulation entre métier, API et smart contract. Le patient soumet une demande depuis le portail ; l'API vérifie le contexte et les accès ; le contrat **EVM** crée la claim ; l'assureur valide et déclenche le paiement si les conditions sont réunies.

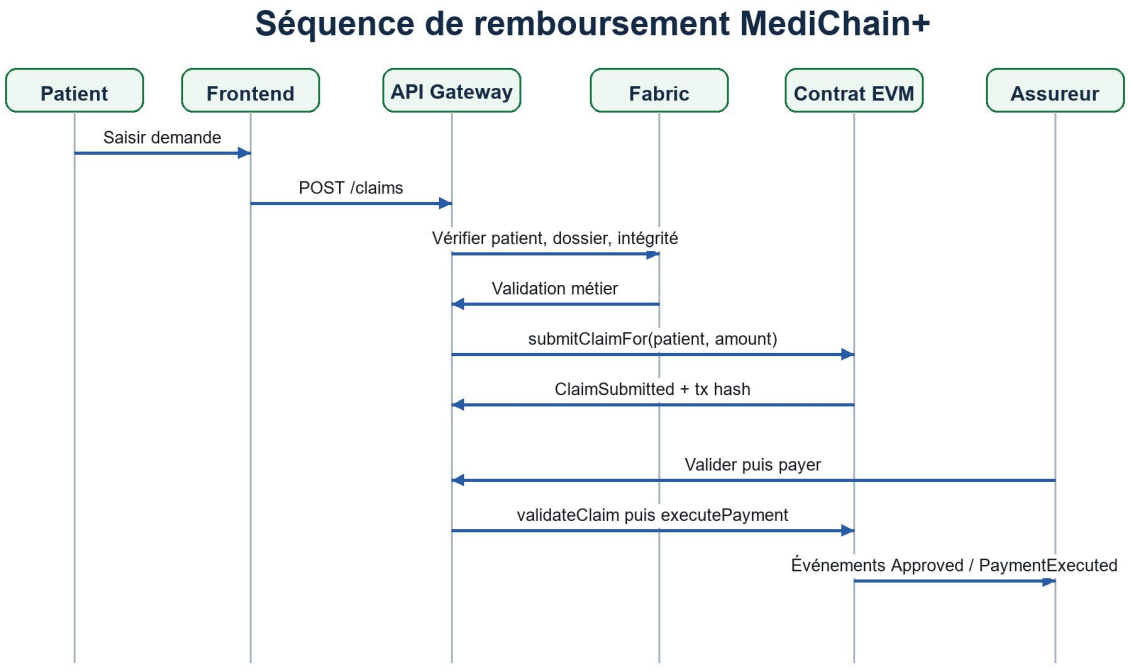


Figure 6. Diagramme de séquence du remboursement.

3.6 Conception de la sécurité

La sécurité n'est pas traitée comme un module isolé. Elle traverse toute l'architecture : authentification wallet ou mode Démo, **JWT**, **RBAC**, contrôle d'accès métier, chiffrement des fichiers, empreintes **SHA-256**, événements blockchain, validation des paramètres et tests automatisés.

Le mode Démo facilite la présentation académique, mais il doit rester compris comme un mode local de démonstration. En production, l'autorisation devrait s'appuyer sur des identités vérifiées, des rôles configurés, une gestion rigoureuse des clés et une gouvernance entre organisations.

3.7 Analyse des Risques et Sécurité

Une solution blockchain de santé ne peut pas être évaluée uniquement par ses fonctionnalités. Elle doit aussi identifier les menaces qui pourraient compromettre la confidentialité, l'intégrité ou la disponibilité des données. **MediChain+** répond à cette exigence par une combinaison de contrôles applicatifs, cryptographiques et contractuels.

Le tableau suivant synthétise les principaux risques du projet et les mesures de mitigation prévues ou déjà démontrées dans le prototype. Certaines mesures sont suffisantes pour une démonstration académique, tandis qu'une version de production devrait renforcer la gestion des clés, la supervision et les politiques inter-organisations.

Tableau 4. Analyse des risques et mesures de mitigation

Risque	Impact	Mitigation dans MediChain+
Accès non autorisé	Consultation de dossiers, prescriptions ou claims par un acteur non habilité.	JWT, RBAC, requireRoles, contrôles d'accès patient/prescription et tests d'accès interdits.
Altération de données	Perte de confiance dans les dossiers, prescriptions ou preuves de remboursement.	Hash SHA-256, historiques Fabric, événements EVM et séparation entre contenu médical et preuve.
Vulnérabilité smart contract	Double paiement, paiement non autorisé ou perte de fonds du contrat.	OpenZeppelin Ownable, ReentrancyGuard, refus des claims invalides et tests de validation/paiement.
Replay attack	Réutilisation d'une preuve ou d'une requête pour rejouer une opération sensible.	Challenge wallet, contexte JWT et validation côté API ; en production, nonce court et expiration stricte à renforcer.

Risque	Impact	Mitigation dans MediChain+
Gestion des clés	Compromission d'un wallet, d'une clé API ou d'une identité Fabric.	Séparation des rôles et configuration .env ; en production, coffre de secrets et rotation des clés.
Menace interne	Usage abusif d'un rôle assureur, médecin ou administrateur.	RBAC, journaux API, événements blockchain et traçabilité Fabric/EVM pour faciliter l'audit.
Exposition IPFS	Accès à un fichier médical si un CID circule hors contexte.	Fichiers chiffrés AES-256-GCM, stockage hors chaîne, conservation de CID/preuves sans données médicales brutes.
Abus API ou déni de service	Dégradation du service ou surcharge des routes de démonstration.	Security headers, limite JSON, rate limiter et validations de paramètres dans l'API Gateway.

La lecture de ce tableau confirme que le risque le plus sensible n'est pas seulement technique. Dans un système de santé, la mauvaise attribution d'un rôle, la fuite d'un CID ou la compromission d'une clé peuvent avoir autant d'impact qu'une erreur de smart contract.

La stratégie retenue consiste donc à combiner plusieurs barrières : contrôle d'accès avant l'appel métier, stockage chiffré hors chaîne, preuves cryptographiques, événements auditables et tests automatisés. Cette défense en profondeur augmente la robustesse du prototype tout en indiquant clairement les points à renforcer pour une version de production.

Chapitre 4 : Technologies utilisées et réalisation

Objectif du chapitre

Ce chapitre détaille les technologies du projet et explique pourquoi chaque composant a été choisi dans **MediChain+**.

4.1 Frontend React et Vite

Le frontend est construit avec **React**, **TypeScript** et **Vite**. Il fournit une landing page, un écran d'authentification, une barre d'état des services et quatre portails. L'intérêt de **React** est de structurer l'interface sous forme de composants réutilisables : StatusBar, ToastContainer, JsonOutput et portails métiers.

Vite accélère le développement local et facilite le build de production. **TypeScript** renforce la lisibilité du code en explicitant les types d'authentification, de réponses API et de rôles. L'interface est pensée pour la démonstration : le mode Démo rend tous les portails accessibles sans dépendre de MetaMask.

4.2 API Gateway Express

L'**API Gateway Node.js/Express** centralise les accès. Elle fournit l'endpoint /health, les routes d'authentification wallet, les routes **Fabric**, les routes **IPFS**, les routes de remboursement et les routes de traçabilité des médicaments.

Cette couche protège les services internes. Elle reçoit les requêtes du frontend, vérifie le **JWT**, contrôle les rôles, valide les paramètres, puis appelle **Fabric**, **EVM** ou **IPFS**. Elle joue aussi un rôle de bridge entre des technologies qui n'ont pas les mêmes modèles d'exécution.

4.3 Hyperledger Fabric

Hyperledger Fabric est adapté aux réseaux permissionnés. Dans **MediChain+**, il correspond aux flux privés : registre patient, dossiers médicaux, consentements et prescriptions. Le projet contient des chaincodes **TypeScript** pour patient-registry, medical-record et prescription.

Fabric permet de raisonner en termes d'organisations connues, de politiques d'accès et d'historique contrôlé. Pendant la validation locale, l'environnement utilisé par le frontend indiquait **Fabric** en mode mock, ce qui permet de démontrer les portails sans démarrer toute l'infrastructure **Fabric**.

4.4 Solidity, Hardhat et OpenZeppelin

La couche **EVM** est développée en **Solidity** avec **Hardhat**. **Hardhat** fournit la compilation, un réseau local, les scripts de déploiement et l'environnement de test. Les contrats sont écrits pour être compatibles avec un déploiement local ou **Polygon**.

OpenZeppelin est utilisé pour des primitives reconnues : Ownable pour le contrôle d'accès propriétaire et ReentrancyGuard pour limiter les risques pendant les paiements. Ces choix évitent de réécrire des mécanismes sensibles à partir de zéro.

4.5 IPFS, chiffrement et hachage

IPFS Kubo est prévu pour stocker les fichiers médicaux hors chaîne. Le principe est de chiffrer le fichier, de l'envoyer à **IPFS**, puis de conserver un CID et une empreinte de contrôle. L'empreinte **SHA-256** permet de vérifier qu'un contenu n'a pas été modifié.

Le chiffrement **AES-256-GCM** apporte à la fois confidentialité et authentification du contenu. **IPFS** ne doit pas être considéré comme un espace de stockage public sans protection ; dans **MediChain+**, le chiffrement est une condition de sécurité indispensable.

4.6 Correspondance technologies / responsabilités

Tableau 5. Correspondance technologies et responsabilités

Technologie	Responsabilité dans le projet
React + TypeScript + Vite	Interface utilisateur, portails métiers, mode Démo, état des services et interaction avec l'API.
Node.js / Express	API Gateway, authentification, RBAC, validation des requêtes et orchestration des services.
Hyperledger Fabric	Données privées : patients, consentements, dossiers médicaux et prescriptions.
Solidity / Hardhat	Smart contracts de remboursement et de traçabilité pharmaceutique ; tests automatisés.
OpenZeppelin	Primitives de sécurité reconnues : Ownable et ReentrancyGuard.
IPFS Kubo	Stockage distribué des fichiers chiffrés, CID, pinning et récupération.

Technologie	Responsabilité dans le projet
JWT / Wallet	Session applicative, preuve de possession de wallet et rôles applicatifs.
Docker / scripts / tests	Environnement de démonstration, validation reproductible et préparation DevOps.

La valeur ajoutée de cette stack vient de sa complémentarité. **React** et **Vite** rendent la démonstration fluide ; **Node.js** et **Express** structurent l'orchestration ; **Hyperledger Fabric** protège les données privées ; **Solidity** et **Hardhat** rendent les règles financières vérifiables ; **OpenZeppelin** réduit les risques sur les primitives sensibles ; **IPFS** sépare les fichiers du registre ; Docker et GitHub Actions préparent la reproductibilité.

4.7 Organisation de l'équipe

- Ilyas Ajelyan : Smart contracts **Solidity**, coordination et logique **EVM**.
- Anas El Karkouri : **Hyperledger Fabric**, chaincodes et intégration **IPFS**.
- Omar Babba : Frontend **React**, portails utilisateurs et expérience de démonstration.
- Omar Gaga : **API Gateway**, identité, **JWT**, **RBAC** et intégration des services.
- Amin Er-Rebbat : Sécurité, DevOps, tests, audit et validation.

Cette répartition respecte l'esprit des documents de cadrage : chaque membre couvre une brique principale et contribue à l'intégration globale. Le projet exige une coordination forte, car les portails frontend, les routes API, les chaincodes, les contrats et les tests dépendent les uns des autres.

Chapitre 5 : Tests, résultats et validation

Objectif du chapitre

Ce chapitre présente l'exécution réelle du projet, les captures live, les résultats de validation, les limites observées et les pistes d'amélioration.

5.1 Environnement exécuté

Le projet testé se trouve dans le dossier local newprojectultra\MediChainPlus. Le frontend **React** était accessible sur localhost:5173. L'**API Gateway** a été lancée sur localhost:3001. La couche **EVM** locale utilisait **Hardhat** avec chainId 31337. Les contrats de remboursement et de traçabilité étaient configurés.

L'endpoint /health a retourné un statut ok pour le service medichain-api-gateway, avec les contrats configurés et **Fabric** détecté. La barre d'état du frontend affichait API ok, **Fabric** mock, **EVM** chain 31337, **IPFS** hors ligne et le nombre de remboursements. Ces informations donnent une preuve visible de l'environnement de démonstration.

5.2 Captures de l'application lancée

Les écrans suivants proviennent des captures Playwright réalisées sur l'application exécutée localement. Ils ne sont pas issus d'anciennes images statiques du dépôt.

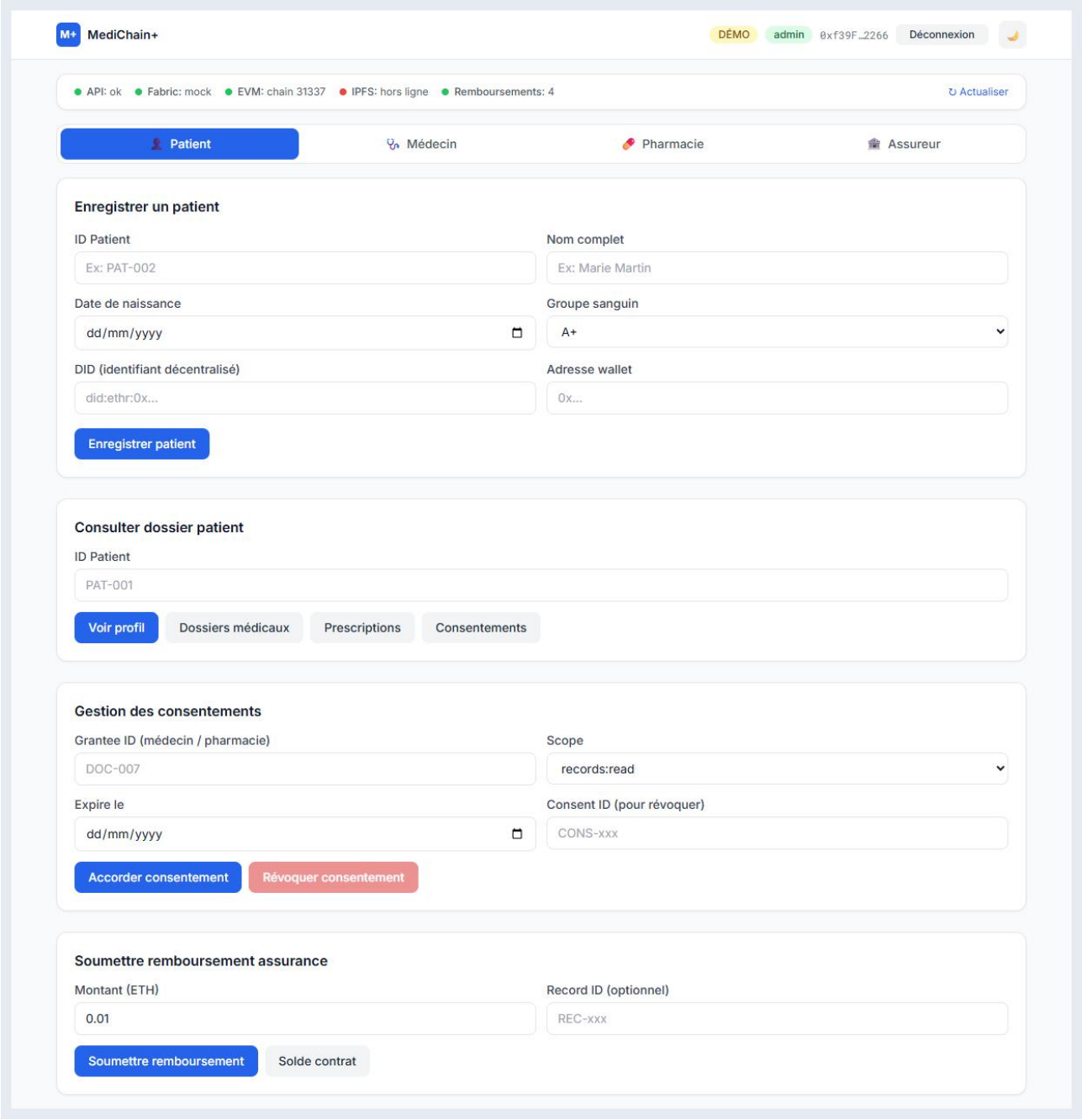


Figure 7. Interface principale après activation du mode Démo.

5.3 Portail Patient

Le portail Patient regroupe les opérations liées au profil, aux consentements, aux prescriptions et à la soumission de remboursement. Il montre comment l'utilisateur final interagit avec une solution blockchain sans manipuler directement les registres distribués.

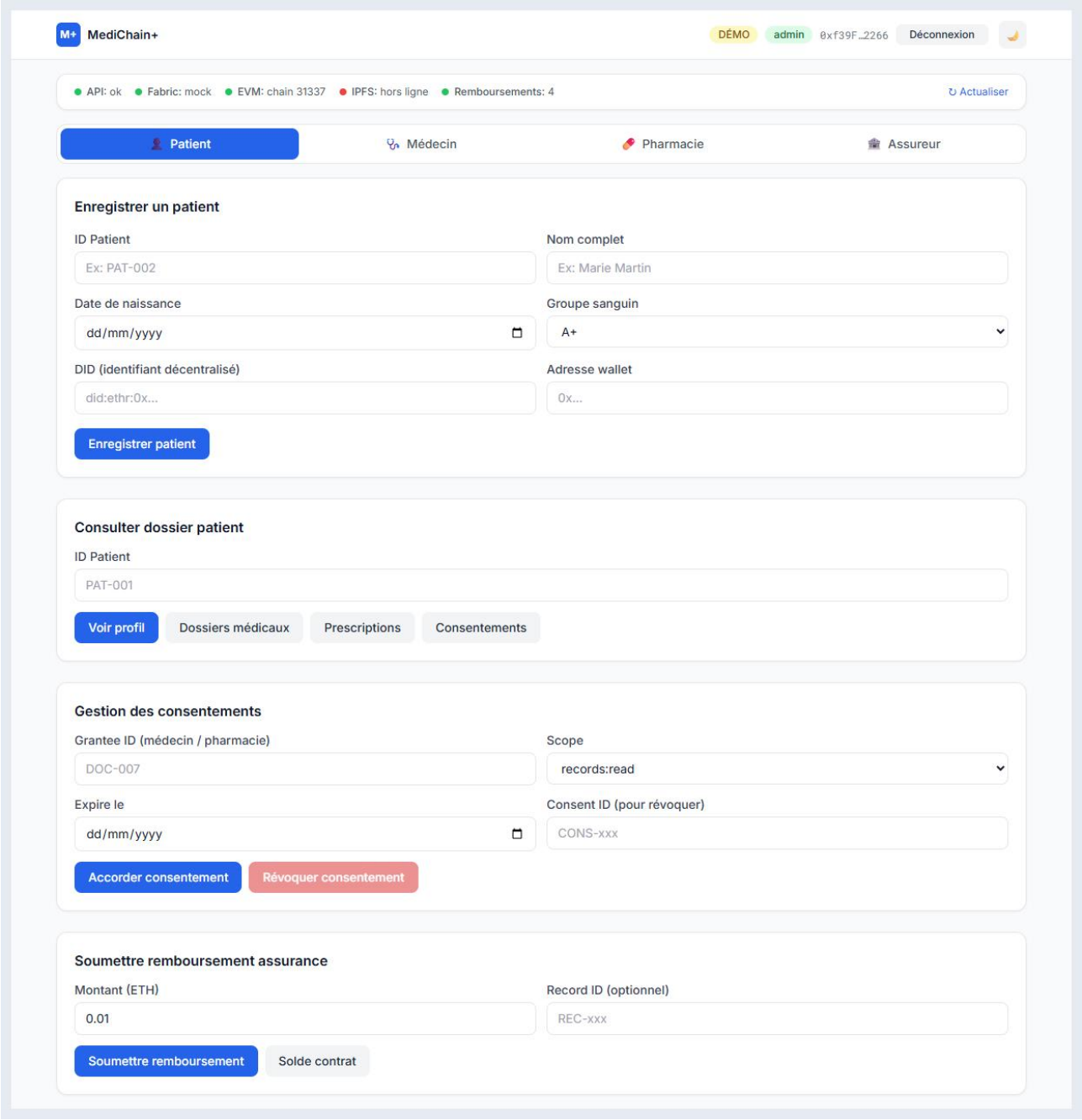


Figure 8. Portail Patient exécuté localement.

5.4 Portail Médecin

Le portail Médecin couvre la recherche de dossiers, la création de dossiers médicaux, la création de prescriptions et la vérification d'intégrité. Il représente la partie clinique du workflow, c'est-à-dire la production de données qui doivent rester fiables et contrôlées.

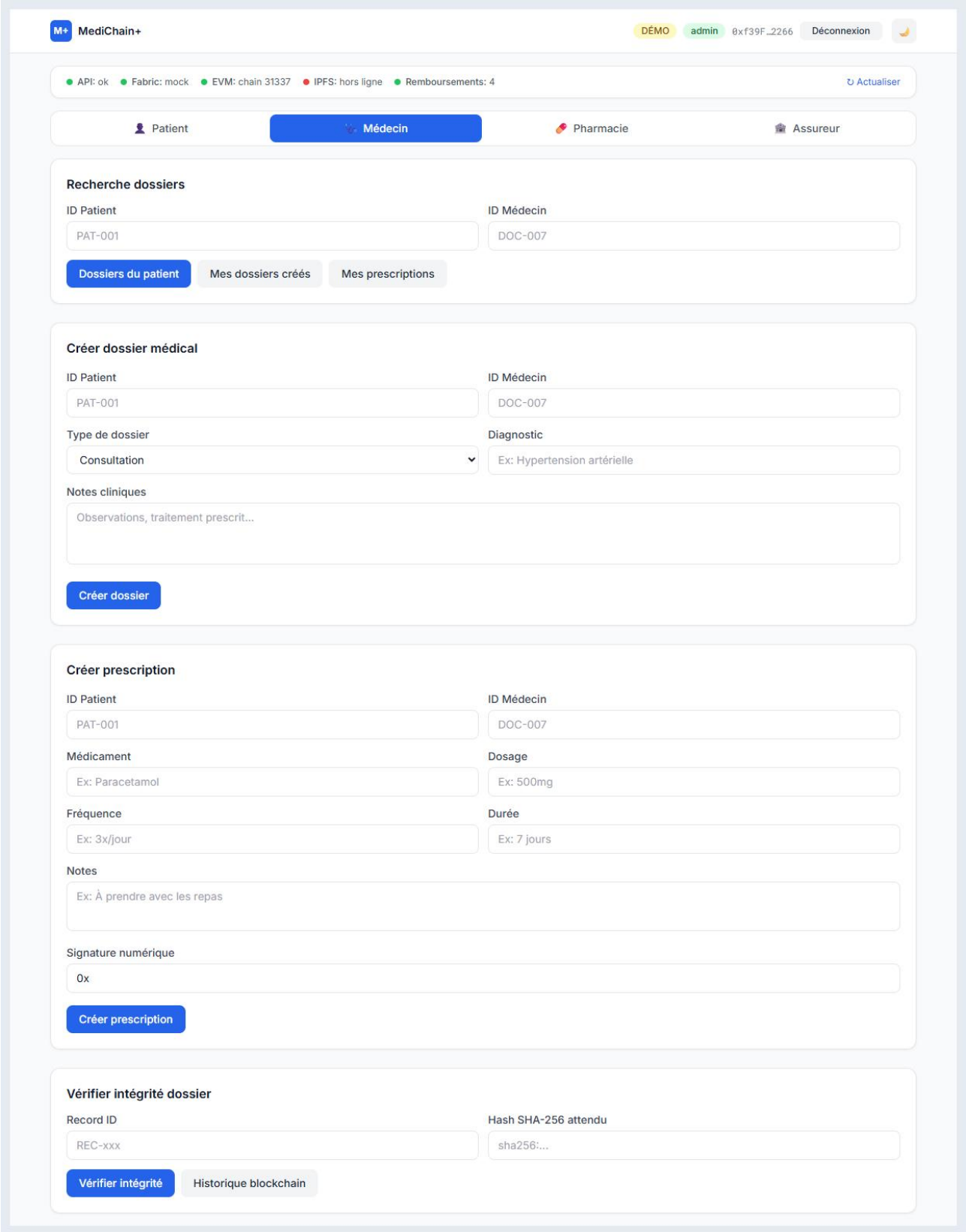


Figure 9. Portail Médecin exécuté localement.

5.5 Portail Pharmacie

Le portail Pharmacie permet de consulter les prescriptions, vérifier les signatures, dispenser un traitement et suivre un lot. Il relie le monde médical à la chaîne pharmaceutique et met en évidence le besoin de traçabilité.

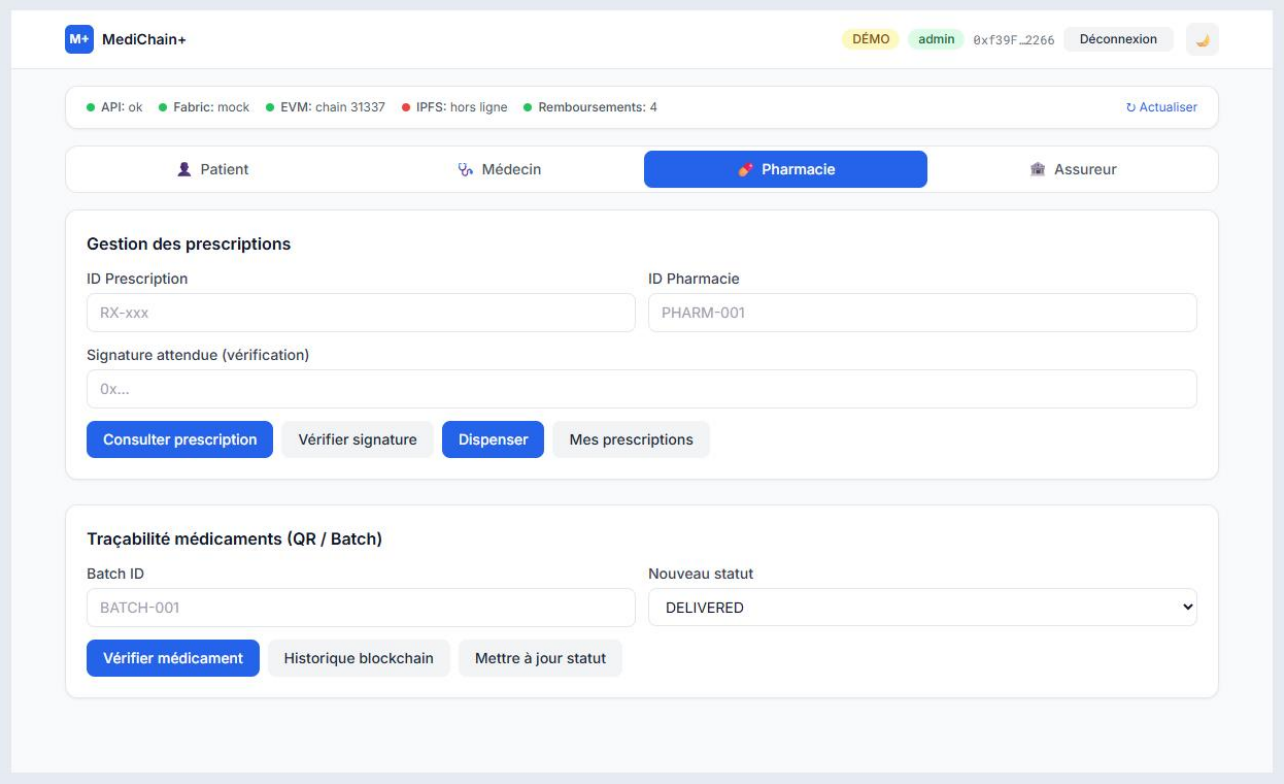


Figure 10. Portail Pharmacie exécuté localement.

5.6 Portail Assureur

Le portail Assureur matérialise le cycle on-chain du remboursement. Il permet de consulter les claims, valider une demande, exécuter le paiement et lire les événements. Cette partie démontre directement l'apport du smart contract.

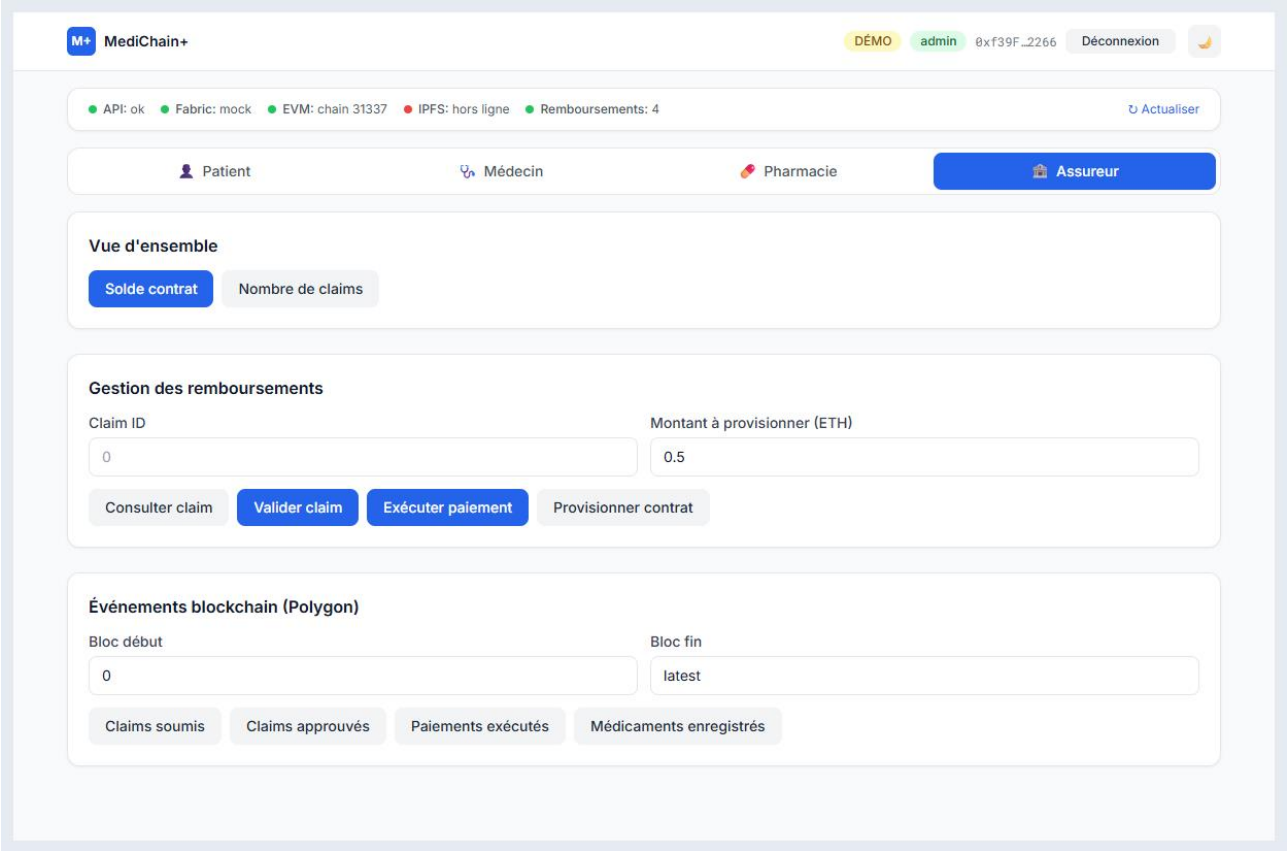


Figure 11. Portail Assureur exécuté localement.

5.7 Résultats de validation

Tableau 6. Résultats de validation technique

Vérification	Résultat observé
Compilation Hardhat	Succès. Les artefacts étaient à jour et les contrats étaient compilables dans l'environnement local.
Tests automatisés	Succès. 47 tests passants couvrant sécurité API, ReimbursementContract et DrugTraceabilityContract.
Build frontend	Succès. Le build React/Vite a été généré sans erreur bloquante.
API /health	Succès. Service medichain-api-gateway ok, contrats configurés, Fabric configuré.
Captures live	Succès. Captures réalisées sur localhost:5173 après activation du mode Démo.
IPFS	Configuré mais hors ligne pendant la capture, car le daemon Kubo n'était pas lancé.

5.8 Indicateurs quantitatifs

Afin de rendre la validation plus objective, plusieurs mesures ont été relevées sur l'environnement local de démonstration. Ces valeurs ne constituent pas un benchmark de production ; elles servent à montrer que les composants principaux répondent correctement dans un contexte contrôlé.

Les tests ont été relancés avec npm test, le build frontend avec npm run build, l'**API Gateway** a été mesurée sur /health, et le cycle **ReimbursementContract** a été mesuré sur **Hardhat** local pour la création, la validation et l'exécution d'une claim.

Tableau 7. Indicateurs quantitatifs de validation

Indicateur	Valeur observée	Interprétation
Tests automatisés	47 tests passants, 605 ms de temps Mocha ; 3,08 s de durée commande.	Couverture des accès API, du remboursement et de la traçabilité.
API /health	12 requêtes HTTP 200 ; moyenne 25,2 ms ; min 9,2 ms ; max 123,6 ms.	Gateway disponible localement et réponse stable sur l'endpoint de santé.
Création de claim	3,8 ms sur Hardhat local.	Soumission de remboursement rapide dans le réseau local de test.
Validation de claim	1,9 ms sur Hardhat local.	Approbation assureur exécutée sans latence significative en local.
Exécution du remboursement	2,3 ms sur Hardhat local.	Paiement on-chain local validé avec événement PaymentExecuted.
Acteurs supportés	5 profils : patient, médecin, pharmacie, assureur, administrateur.	Les rôles métier principaux sont couverts par les portails et le RBAC.
Workflows visibles	15 actions métier structurées dans le diagramme de cas d'utilisation.	Le prototype couvre les opérations principales de chaque acteur.
Build frontend	4,35 s de durée commande ; Vite indique 1,70 s ; 44 modules transformés.	Interface générable en production sans erreur bloquante.

Tableau 8. Comparaison du stockage on-chain et hors chaîne

Approche	Avantages	Limites dans un contexte santé
Stockage entièrement on-chain	Auditabilité forte et état partagé immédiatement vérifiable.	Coût, exposition publique, faible confidentialité et mauvaise adaptation aux fichiers médicaux.
Hyperledger Fabric	Réseau permissionné, acteurs connus, métadonnées privées et politiques d'accès.	Nécessite gouvernance multi-organisations et configuration plus lourde qu'un simple registre public.
IPFS chiffré	Stockage distribué des fichiers, CID vérifiable, séparation des contenus lourds.	Le CID ne suffit pas à protéger un fichier ; le chiffrement et le contrôle d'accès restent indispensables.
Approche hybride MediChain+	Données sensibles hors chaîne, preuves vérifiables, remboursements automatisés et meilleure séparation des responsabilités.	La production demanderait supervision, rotation des clés, Kubo actif et politiques d'endorsement complètes.

5.9 Analyse des résultats obtenus

Les résultats montrent que **MediChain+** est cohérent avec son objectif académique. Le projet ne se limite pas à une interface ; il comprend des contrats, des routes API, des portails, une logique de sécurité, des tests et une architecture multi-composants. Les captures live renforcent la crédibilité de la partie réalisation.

La validation confirme aussi l'intérêt de séparer les couches. Les tests des contrats sécurisent les règles on-chain, les tests API contrôlent les autorisations, et le build frontend vérifie que l'interface peut être générée. Cette combinaison est plus solide qu'une simple démonstration manuelle.

Les limites restent importantes à mentionner. **IPFS** doit être lancé pour démontrer pleinement le stockage distribué. **Fabric** en mode mock facilite la présentation mais ne remplace pas une validation complète multi-organisations. Enfin, une version de production devrait renforcer la gestion des clés, la conformité réglementaire, la journalisation et la supervision.

5.10 Perspectives

- Lancer **Kubo** pendant la démonstration afin de montrer l'upload, la récupération CID et la vérification de fichiers réels.

- Déployer **Fabric** en réseau multi-organisations avec politiques d'endorsement explicites.
- Tester les contrats sur un testnet **Polygon** et documenter les coûts de transaction.
- Ajouter des QR codes pour les lots pharmaceutiques et relier l'interface Pharmacie au scan.
- Renforcer l'audit de sécurité avec Slither, tests de charge API et scénarios d'abus.

Conclusion générale

MediChain+ démontre l'intérêt d'une architecture blockchain hybride dans un contexte de santé numérique. Le projet répond à trois difficultés majeures : la fragmentation des dossiers médicaux, la lenteur des remboursements et le besoin de traçabilité pharmaceutique. Il montre aussi que la blockchain doit être utilisée avec discernement : toutes les données ne doivent pas être placées on-chain.

La solution combine **Fabric**, **EVM**, **IPFS**, **API Gateway** et **React**. Cette combinaison permet de séparer les données sensibles, les preuves cryptographiques, les règles automatisées et l'interface utilisateur. Le patient, le médecin, la pharmacie et l'assureur disposent chacun d'un portail adapté à leurs responsabilités.

Le projet a été validé par exécution locale, captures live, compilation, tests automatisés, build frontend et mesures quantitatives. Les résultats montrent que le livrable est démontrable et techniquement cohérent. L'analyse des risques complète cette validation en reliant les mécanismes **JWT**, **RBAC**, **SHA-256**, **AES-256-GCM**, **Fabric**, **IPFS** et smart contracts aux menaces réelles du domaine santé.

Sur le plan pédagogique, **MediChain+** illustre plusieurs notions du module Blockchain & Cryptographie : hachage **SHA-256**, chiffrement **AES-256-GCM**, signatures, identité wallet, registres distribués, smart contracts, immutabilité, **RBAC** et auditabilité. Le projet constitue donc une application concrète des concepts étudiés.

Références Bibliographiques

★ **Kuo, Kim & Ohno-Machado (2017) :** Blockchain distributed ledger technologies for biomedical and health care applications, JAMIA, 24(6), 1211-1220, DOI: 10.1093/jamia/ocx068.

★ **Azaria, Ekblaw, Vieira & Lippman (2016) :** MedRec: Using Blockchain for Medical Data Access and Permission Management, 2nd International Conference on Open and Big Data, DOI: 10.1109/OBD.2016.11.

★ **Androulaki et al. (2018) :** Hyperledger Fabric: A Distributed Operating System for Permissioned Blockchains, EuroSys 2018, DOI: 10.1145/3190508.3190538.

★ **Zhang et al. (2018) :** FHIRChain: Applying Blockchain to Securely and Scalably Share Clinical Data, Computational and Structural Biotechnology Journal, 16, 267-278, DOI: 10.1016/j.csbj.2018.07.004.

★ **McGhin et al. (2019) :** Blockchain in healthcare applications: Research challenges and opportunities, Journal of Network and Computer Applications, DOI: 10.1016/j.jnca.2019.02.027.

★ **Esposito et al. (2018) :** Blockchain: A Panacea for Healthcare Cloud-Based Data Security and Privacy?, IEEE Cloud Computing, 5(1), 31-37, DOI: 10.1109/MCC.2018.011791712.

★ **Dépôt GitHub du projet :** https://github.com/omarbabba779xx/MediChainPlus-EMSI

★ **Projet local exécuté :** C:\Users\omarb\OneDrive\Documents\newprojectultra\MediChainPlus

★ **Cahier des charges :** MediChain+ EMSI, document fourni.

★ **Schéma d'exécution :** MediChain+ Project Execution Schema & Team Distribution Plan, document fourni.

★ **Architecture :** MediChain Architecture, document fourni.

★ **Hyperledger Fabric :** https://hyperledger-fabric.readthedocs.io/

★ **Solidity :** https://docs.soliditylang.org/

★ **Hardhat :** https://hardhat.org/docs

★ **OpenZeppelin Contracts :** https://docs.openzeppelin.com/contracts/

★ **IPFS Kubo :** https://docs.ipfs.tech/

★ **W3C DID Core Specification :** https://www.w3.org/TR/did-core/

★ ★ ★